1 Introduction

The objective of this project was to explore black hole structure in general relativity by numerically integrating a system of six differential equation. Through three main computational experiments—Einstein ring formation, unstable spherical photon orbits, and accretion disk ray tracing—we developed, validated, and applied a custom Runge–Kutta integration framework to model how light propagates near black holes.

2 Einstein Ring in Schwarzschild Spacetime

2.1 Objective

The goal of this task was to test and validate our numerical integration scheme by determining whether a photon, launched from a given initial position near a Schwarzschild black hole, would return to its original point. This served as a verification of both the stability and accuracy of our Runge–Kutta integration framework.

2.2 Methodology

We implemented this in `einstein_ring.cpp`. The metric components and their derivatives were initialized in `metric.h`, and we evolved the system of six first-order differential equations as described in the project handout. We set the black hole's mass $M = 1$ and spin $a = 0$ since the Schwarzschild metric corresponds to a non-rotating black hole.

Our initial photon conditions were defined as:

$$r = 10M, \quad \theta = \frac{\pi}{2}, \quad \phi = 0, \quad u_r = -\sqrt{g_{rr}}\cos\xi, \quad u_\theta = 0, \quad u_\phi = \sqrt{g_{\phi\phi}}\sin\xi$$

where ξ is the critical launch angle provided in the project instructions.

We implemented a custom stopping condition for the integration:

- The photon's ϕ exceeded its starting value by 2π or 4π,
- The integration time exceeded 10,000,
- The photon radius dropped below $2.01M$, close to the event horizon ($2M$).

These conditions ensured that the integration terminated if the photon became trapped, escaped to infinity, or entered the near-horizon region where the equations break down.

We used our adaptive RK45 Dormand–Prince integrator (`rk45_doormand_prince.h`) to evolve the equations. After integration, the photon trajectory was plotted in r–ϕ space using Cartesian coordinates:

$$x = r\cos\phi, \quad y = r\sin\phi$$

This plotting was implemented in `einstein_ring.py`. See Figure 1.

2.3 Interpolation and Error Estimation

To determine whether the photon returned exactly to its starting point, we interpolated the trajectory data and used Newton–Raphson iterations to find the precise time t at which ϕ matched the initial angle. We employed the dense-output polynomial from our RK45 solver and solved:

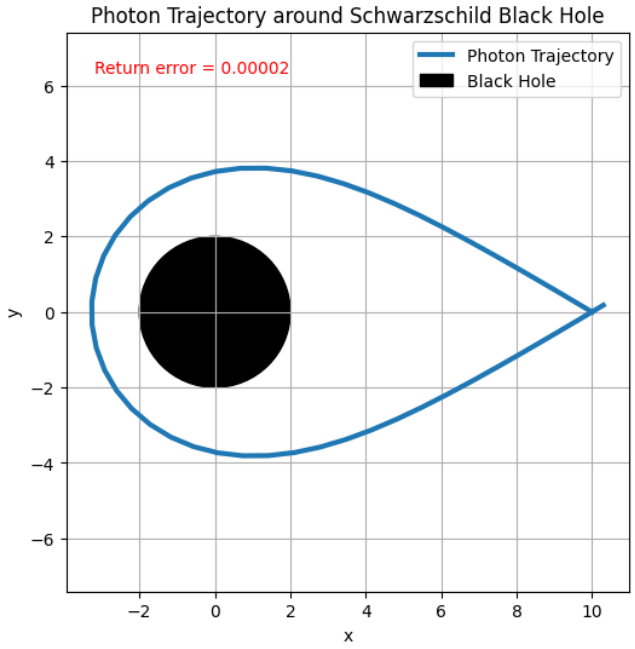
$$f(t + \theta * h) - \phi_0 = 0$$

The `newton_raphson.h` file handled these iterations. We used initial guesses of 0.2, 0.5, and 0.8 to ensure convergence.

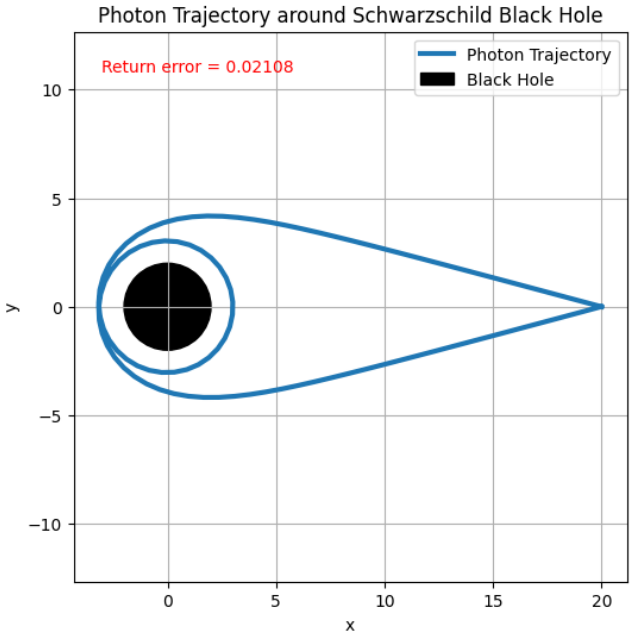
Once t was found, we used the radial interpolation polynomial and computed the deviation from the initial radius. For our first orbit, the radial error was approximately 1.6×10^{-5}, while for the second orbit, the error increased to around 0.021. The higher error in the second orbit is expected because the photon completed an additional loop around the black hole, allowing more numerical error to accumulate.

2.4 Interpretation

Both results demonstrate that the integrator performs accurately for photon trajectories over large time intervals. The small fractional errors indicate that the integrator maintains stability and precision across multiple orbits. Further accuracy could be achieved by tightening the relative and absolute tolerances or employing a higher-order adaptive method such as Dormand–Prince 8(7).



(a) $\xi = 0.48857874$



(b) $\xi = 0.24904964$

Figure 1: Trajectory of the Einstein ring for two observer angles: left shows $\xi = 0.48857874$, right shows $\xi = 0.24904964$.

3 Unstable Photon Orbits

3.1 Objective

This section aimed to simulate a series of unstable spherical photon orbits around a maximally spinning Kerr black hole ($a = 1$) and examine how small perturbations grow over time. We analyzed both the 3D trajectories and the growth of the fractional radius error to quantify orbital instability.

3.2 Methodology

The simulation was implemented in `unstable_orbit.cpp`. The class `boyer_lindquist_metric` computes all relevant 3+1 metric quantities (α, β^i, and γ^{ij}) and their derivatives. These were used to construct the right-hand-side function `make_rhs(metric)`, which returns the time derivatives of:

$$(r, \theta, \phi, u_r, u_\theta, u_\phi)$$

according to Eqs. (20)–(25) of the project handout.

Initial conditions were taken from Table 1. Each orbit starts with $u_r = 0$, $\theta = \pi/2$, and $\phi = 0$, ensuring the photon begins on the equatorial plane. Integration used RK45 with tolerances of 10^{-12}, stopping when:

- $r < 1.01\, r_h$ (approaching the event horizon),
- $r > 200$ (escaping to infinity), or
- $t > 400$.

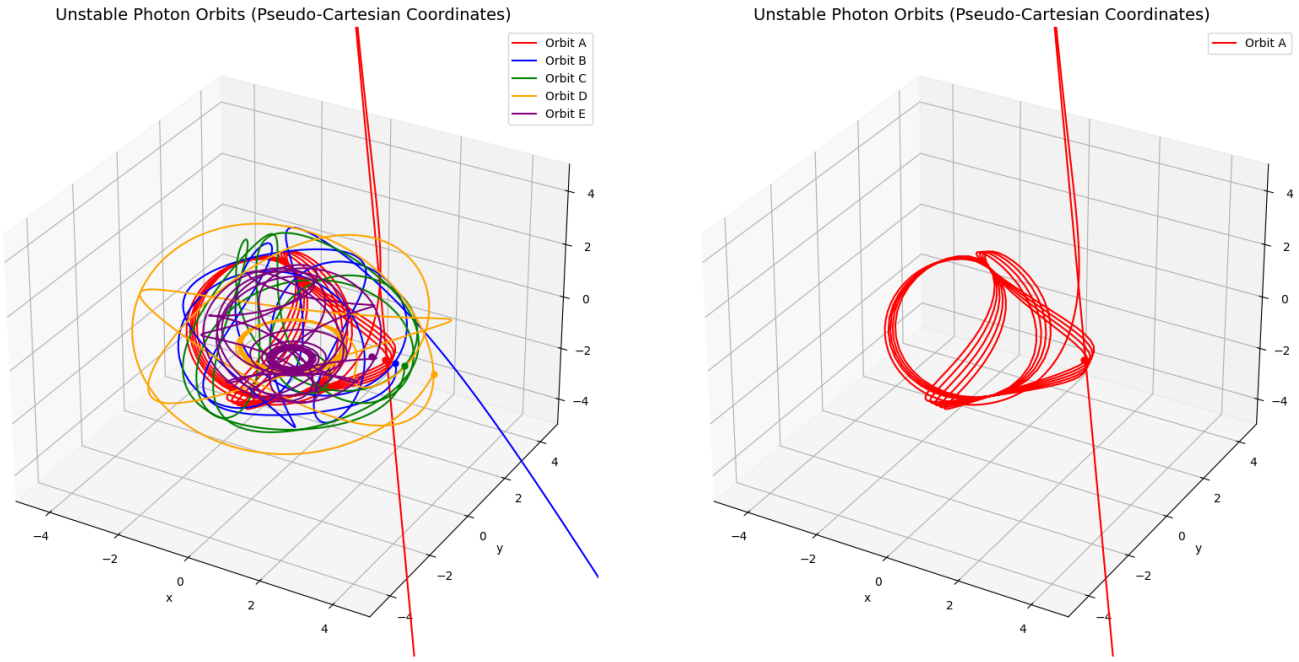
The trajectories were then converted to pseudo-Cartesian coordinates:

$$x = \sqrt{r^2 + a^2}\sin\theta\cos\phi, \quad y = \sqrt{r^2 + a^2}\sin\theta\sin\phi, \quad z = r\cos\theta$$

and plotted in 3D.

3.3 Results and Discussion

The 3D trajectory plot shows that orbits B, C, and E diverge rapidly from the initial spherical radius, while A and D remain quasi-stable longer before becoming unstable.



(a) All Trajectories **(b)** Trajectory A

Figure 2: Unstable Photon Orbit Trajectories

To quantify this, we computed the fractional radius error:

$$\text{frac_err} = \frac{|r - r_0|}{r_0}$$

and plotted it as a function of time.

For Orbit C, the fractional error grows exponentially around $t \approx 25$ and becomes visibly unstable by $t \approx 180$. This occurs earlier than in the reference example (instability near $t \approx 125$), likely due to different numerical step-size controls.

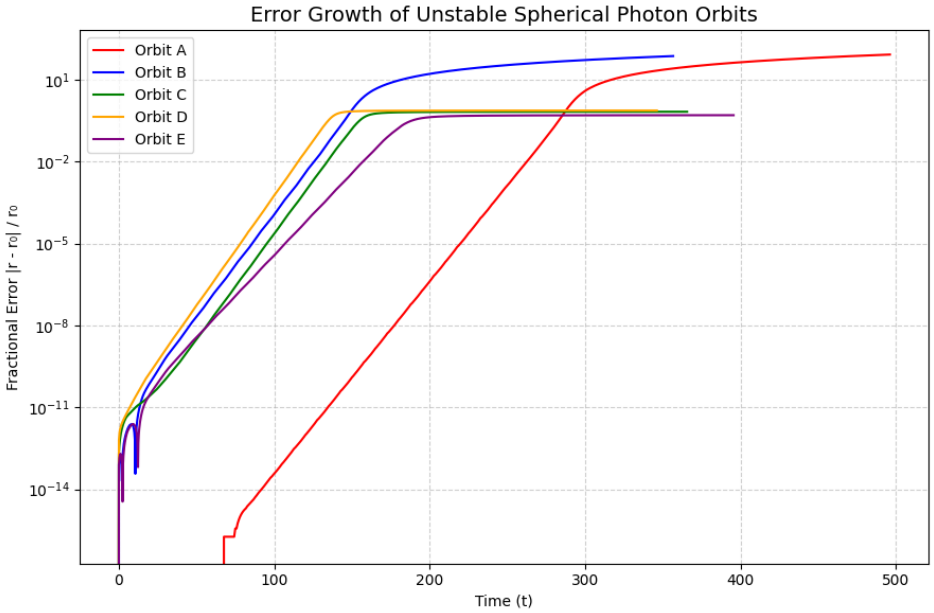


Figure 3: Fractional Error Over Time

Overall, the results confirm that our integrator reproduces the correct qualitative behavior of unstable spherical photon orbits. The exponential growth in fractional radius error characterizes the chaotic instability near the photon sphere. The differing instability times across orbits highlight the sensitivity of photon dynamics to small

perturbations.

4 Ray Tracing of Accretion Disks

4.1 Objective

The final section extended our integrator to simulate ray tracing of photons emitted from a infinitely thin accretion disk around a black hole, producing a realistic image as seen by a distant observer.

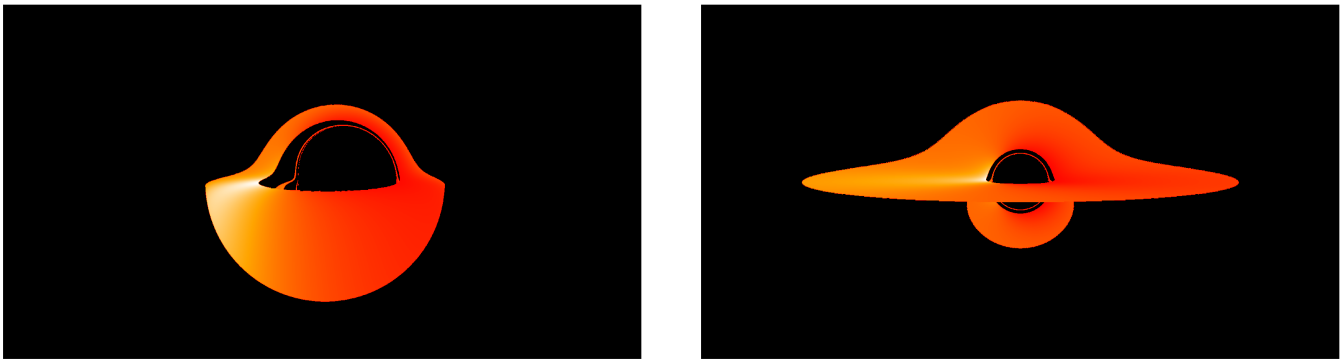
4.2 Methodology

This task was implemented in `ray_trace.cpp`, which calls `accretion.h` for determining disk intersections and computing intensities.

The disk was modeled as a thin equatorial ring ($\theta = \pi/2$). Each pixel on the observer's screen corresponds to impact parameters (α, β), defining photon direction. Each photon was integrated backward until:

1. The photon escaped ($r > r_{\text{observer}}$),
2. Fell into the black hole ($r < 1.01\, r_h$),
3. Intersected the disk ($r_{\text{in}} < r < r_{\text{out}}$) with $(\theta - \pi/2)$ changing sign.

If a photon hit the disk, interpolation and Newton–Raphson methods were used to find the exact crossing time, after which we computed the local intensity (implemented in `hit_accretion()`). This interpolation was implemented the same way we found the final radius for the Einstein rings. To prevent numerical artifacts near the image center, photons with near 0 angular momentum were given a slight inward velocity. Intensities were normalized to 8-bit values and rendered in `raytracer.py` using a color scale from black → red → orange → white.



(a) Maximally Spinning Black Hole viewed from $\theta = 85$ and D =10M

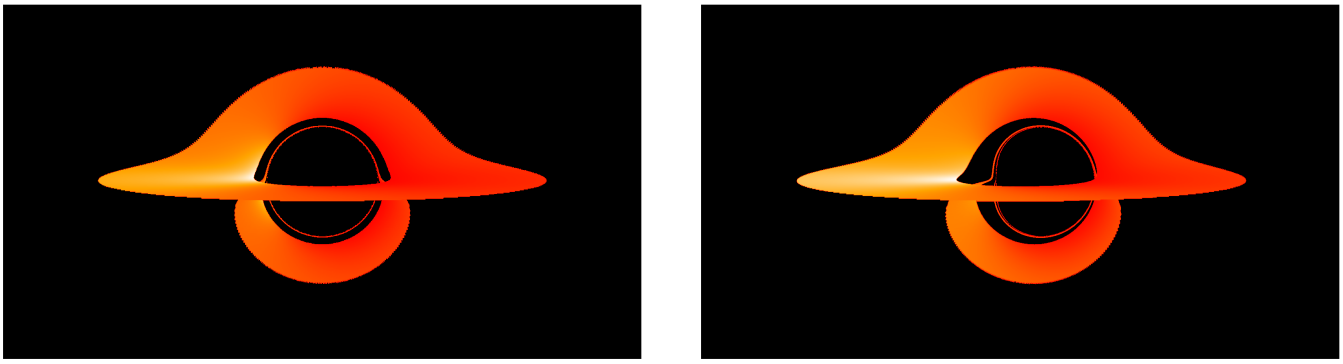
(b) Non Spinning Black Hole viewed from $\theta = 85$, with outer radius of 40M

Figure 4: Example Raytraced Images

4.3 Results

4.3.1 Photon Rings: Spinning vs. Non-Spinning Black Holes

To analyze the effect of spin, we ray traced images for non-spinning ($a = 0$) and maximally spinning ($a = 1$) black holes.



(a) Non Spinning Black Hole

(b) Maximally Spinning Black Hole

Figure 5: Comparison of Black holes based on spin, viewed from $\theta = 85$ and D =500M

In the non-spinning case, the photon ring is circular and symmetric. In the spinning case, the ring shifts and warps to one side, illustrating frame dragging. The ring is no longer spherical either. The region of greatest brightness corresponds to photons approaching from the direction of black hole rotation.

4.3.2 Event Horizon Shape

We visualized the event horizon by setting the disk's inner radius to 2.02 times the event horizon and viewing at $\theta\circ = 90°$.



(a) Non Spinning Black Hole **(b)** Maximally Spinning Black Hole

Figure 6: Comparison of Black holes based on spin, viewed from $\theta = 90$ and D =500M

4.4 Results

For $a = 0$, the horizon appears spherical, while for $a = 1$, it becomes asymmetric and flattened due to rotational effects. The distortion grows with spin, consistent with the theoretical shape of the Kerr horizon.

4.4.1 Modified Intensity Equation

Replacing the standard intensity equation with the final angle ϕ provided a visualization of how photons map from the disk to the observer.



(a) $a = 0$ **(b)** $a = 0.5$ **(c)** $a = 1$

Figure 7: Disk intensity maps for increasing black hole spin. Taken at $\theta = 5$

As spin increased, brightness across the disk became more uniform, indicating that frame dragging distributes photon trajectories more evenly across azimuthal angles. The resulting images reveal the complex mapping between the observer's screen and the rotating disk.

5 Parallelization

5.1 Parallelization Strategy

For the parallelization, we used a design that maps naturally onto both OpenMP and C++ standard threads. The main idea is that each image row can be computed independently, because every pixel requires tracing its own null geodesic through the Kerr spacetime, and no ray interacts with any other.

Our strategy, as stated in the PDF, exploits this "embarrassingly parallel" structure by placing all of the ray-tracing work for a contiguous block of rows into a single reusable function: `render_rows` (located in `src/bh_image.cpp`). Each of the threads that enters this function allocates its own `BoyerLindquistMetric` instance, geodesic right-hand-side (RHS) integrator, and hit counters. Because all these objects are thread-local, no mutable state is shared, and almost all synchronization overhead is eliminated. The only shared state consists of some atomic counters that record the aggregate ray outcomes. These are updated using `fetch_add` operations, which helps ensure that the synchronization costs remain basically negligible.

The code uses OpenMP and enables it automatically through the `-fopenmp` flag, as set in the Makefile. The OpenMP branch defines a parallel region and divides the image rows evenly among worker threads. The work then

proceeds with minimal coordination because each thread simply calls `render_rows` on its assigned subset.

For systems that cannot run OpenMP, the code falls back to a portable implementation using `std::thread`. It checks with `std::thread::hardware_concurrency()` to determine an appropriate level of parallelism, partitions the image into row-chunks, and spawns one worker thread per chunk. Because both back-ends call the same underlying worker lambda, the behavior is identical across both platforms. A similar structure is mirrored in `ray_trace.cpp`.

6 Methods Summary

File	Description
`metric.h`	Computes metric components and derivatives as functions of r and θ.
`einstein_ring.cpp`	Tests the integrator using the critical angle $\xi = 0.48857874$.
`einstein_ringb.cpp`	Tests the integrator using $\xi = 0.24904964$.
`unstable_orbit.cpp`	Integrates unstable spherical photon orbits around a Kerr black hole.
`rk45_dromandprince.h`	Adaptive Runge–Kutta integrator used across tasks.
`rk45_dormand_prince.h`	Adaptive Runge–Kutta integrator used across tasks.
`rk45_doormand_prince.h`	Adaptive Runge–Kutta integrator used across tasks.
`accretion.h`	Determines whether a photon hits the disk and computes its intensity.
`ray_trace.cpp`	Performs ray tracing from the observer's screen into the black hole region.
`einstein_ring.py`	Plots trajectories for Einstein ring tests.
`raytracer.py`	Converts intensity arrays to visual images.

7 Conclusion

Through this project, we successfully implemented a complete numerical framework for photon trajectory integration and visualization in general relativity. Beginning with Einstein Ring tests of our RK45 integrator, we verified its precision and stability by reproducing Einstein ring behavior with minimal numerical error. We moved to unstable orbit testing for spinning black holes, where we observed exponential growth of the fractional radius error.

Finally, ray tracing around spinning and non-spinning black holes produced physically realistic images, including the characteristic bright crescent and asymmetric photon ring of the Kerr black hole. These results qualitatively match theoretical predictions and observations from simulations such as those by the Event Horizon Telescope.

Overall, the project validated our numerical approach and deepened our understanding of light propagation in curved spacetime. Future work could involve implementing higher-order integration methods, or including random perturbations in the accretion disk.